



Math Prerequisites for Data Science

Percentages, Ratios, Powers, Roots, Logarithms & Reading Notation

Session 17

CodeByPaxto: The AI & ML Playground

Hafiz Waseem Ahmed (Paxto)



What We're Covering Today



Percentages, percentage change, ratios, and proportions



Powers, roots, and scientific notation — handling very big and very small numbers



Logarithms, intuitively — what they are, and why they matter later



Reading math notation — Σ , subscripts, function notation, and common symbols



Today's tasks and a mini project — decoding a real data science result



Percentages, Ratios & Proportions

The numbers you'll see in almost every dataset, every metric, and every result from here on



What Is a Percentage, Really?



**"Per
hundred,"
literally**

- Percent literally means "per hundred" — 25% is simply another way of writing 25 out of every 100, or the fraction $25/100$
- A percentage is always a comparison — some part, measured against some whole. Without a clear "whole," a percentage is meaningless on its own
- "90% accuracy" only means something once you know the whole: 90 correct predictions out of 100 total attempts
- Every percentage in this course — accuracy, confidence, growth — is this exact same idea, just applied to a different part and whole



Calculating a Percentage

- The core formula never changes: divide the part by the whole, then multiply by 100 to express it as "per hundred"
- This single formula covers test scores, discounts, model accuracy, survey results — anything expressed as a percentage
- Converting back is just as common: to turn a percentage into a usable decimal, divide by 100

$$\text{percentage} = (\text{part} / \text{whole}) \times 100$$

Example: 42 correct answers out of 50 questions

$$\text{percentage} = (42 / 50) \times 100$$

$$= 0.84 \times 100$$

$$= 84\%$$

Converting a percentage back to a decimal:

$$84\% \rightarrow 84 / 100 \rightarrow 0.84$$



Percentage Change — Increase and Decrease

- Percentage change measures how much something moved, relative to where it started — not just the raw difference
- A jump from 10 to 20 and a jump from 1000 to 1010 are both "+10" in raw terms, but wildly different in percentage terms
- This distinction is exactly why data scientists almost always report percentage change, not raw change, when comparing results

$$\text{percentage change} = ((\text{new} - \text{old}) / \text{old}) \times 100$$

Example: model accuracy improved from 72% to 84%

$$\text{change} = ((84 - 72) / 72) \times 100$$

$$= (12 / 72) \times 100$$

$$\approx 16.7\%$$

A negative result means a decrease, not an error



Real-World Use: Percentages in Data & AI

- Model accuracy is, quite literally, a percentage — correct predictions divided by total predictions
- A confusion matrix's precision and recall are also percentages underneath — you'll compute both directly from this exact formula
- When a dataset report says "12% missing values," it's telling you the part-to-whole ratio of empty cells to total cells

Model accuracy is a percentage in disguise

$\text{accuracy} = (\text{correct_predictions} / \text{total_predictions}) \times 100$

Example: 940 correct out of 1000 predictions

$\text{accuracy} = (940 / 1000) \times 100 = 94\%$

"12% missing values" in a dataset of 500 rows:

$\text{missing_rows} = 0.12 \times 500 = 60 \text{ rows}$



What Are Ratios and Proportions?



**Comparing
two things
directly**

- A ratio compares two quantities directly — a 3:1 ratio of cats to dogs means for every 3 cats, there is 1 dog
- A proportion is simply a statement that two ratios are equal — it's how you scale a ratio up or down while keeping the same relationship
- Ratios don't need a "whole" the way percentages do — they just compare parts to each other directly
- You already use ratios constantly — a recipe's ingredient ratio, a map's scale, or splitting a bill in a certain proportion



Working With Ratios & Proportions

- Ratios are usually simplified the same way fractions are — divide both sides by their greatest common factor
- Cross-multiplication is the standard way to solve a proportion when one value is unknown
- This exact technique is what you'll use to scale a recipe, resize an image proportionally, or split a dataset into train/test sets

Simplifying a ratio

$20 : 30 \rightarrow \text{divide both by } 10 \rightarrow 2 : 3$

Solving a proportion (cross-multiplication)

$$3 / 4 = x / 20$$

$$3 \times 20 = 4 \times x$$

$$60 = 4x$$

$$x = 15$$



Real-World Use: Ratios in Data Science

- An 80:20 train/test split is a ratio — 80% of your data trains the model, 20% evaluates it
- Precision and recall, two of the most common ML evaluation metrics, are ratios — correct positive predictions compared against different totals
- A class imbalance report like "1:99" tells you immediately that one class is dramatically rarer than the other — critical to know before training a model

An 80:20 train/test split on 1,000 rows

$\text{train_rows} = 0.80 \times 1000 = 800$

$\text{test_rows} = 0.20 \times 1000 = 200$

Precision: correct positives out of ALL predicted positives

$\text{precision} = \text{true_positives} / (\text{true_positives} + \text{false_positives})$

Recall: correct positives out of ALL actual positives

$\text{recall} = \text{true_positives} / (\text{true_positives} + \text{false_negatives})$



Practice Set — Percentages & Ratios

Try these yourself first — that's the point

- 1 Out of 250 survey responses, 180 said yes. Calculate the percentage that said yes
- 2 A product's price rose from \$40 to \$52. Calculate the percentage increase
- 3 Simplify the ratio 45:60 to its lowest terms
- 4 A dataset has 5,000 rows split in an 70:30 train/test ratio — calculate how many rows are in each set
- 5 If precision = 0.85 and there were 100 predicted positives, how many were true positives?



Powers, Roots & Scientific Notation

The shorthand mathematicians use so nobody has to write out very large or very small numbers by hand



What Is an Exponent (Power)?



**Repeated
multiplication,
compressed**

- An exponent is shorthand for repeated multiplication — 2^4 simply means "multiply 2 by itself 4 times," not "multiply 2 by 4"
- The base is the number being multiplied; the exponent tells you how many times to multiply it by itself
- Squaring a number (raising it to the power of 2) is one of the most common operations in statistics — it shows up the moment you measure how spread out data is
- Exponents grow explosively fast — this is exactly why they're the natural tool for describing rapid growth, compounding, or exploding error



Rules of Exponents Worth Knowing

- These handful of rules cover the vast majority of exponent situations you'll actually run into
- A negative exponent doesn't mean a negative number — it means "flip it into a fraction"
- Anything raised to the power of 0 is always 1 — a rule that surprises people the first time, but holds with no exceptions

Multiplying same base: add the exponents

$$x^2 \times x^3 = x^5$$

Power of a power: multiply the exponents

$$(x^2)^3 = x^6$$

Negative exponent: flips into a fraction

$$x^{-2} = 1 / x^2$$

Anything to the power of 0

$$x^0 = 1$$



What Is a Square Root (and Cube Root)?



**Undoing
a power,
exactly**

- A square root asks the reverse question of squaring: "what number, multiplied by itself, gives me this?" $\sqrt{25}$ asks that about 25, and the answer is 5
- A cube root asks the same question, three levels deep: "what number, multiplied by itself three times, gives me this?"
- Square roots and squaring are inverse operations — one undoes the other, exactly like addition and subtraction undo each other
- This "undoing" relationship is precisely why standard deviation (which involves a square root) is used to reverse the effect of variance (which involves squaring)



Roots as Fractional Exponents

- Every root can be rewritten as a fractional exponent — this is genuinely how calculators and code compute roots internally
- A square root is the same as raising something to the power of one-half; a cube root is the power of one-third
- Recognizing this connection means you never need to memorize roots and exponents as two separate, unrelated topics

$$\sqrt{x} = x^{(1/2)}$$

$$\sqrt[3]{x} = x^{(1/3)}$$

Example

$$\sqrt{16} = 16^{(1/2)} = 4$$

$$\sqrt[3]{27} = 27^{(1/3)} = 3$$

In Python, this is exactly what you'll type:

```
16 ** 0.5 # 4.0
```

```
27 ** (1/3) # 3.0
```




Real-World Use: Why Squares & Roots Matter in Stats

- Variance measures spread by squaring every deviation from the mean — squaring guarantees every value becomes positive, so distances don't cancel out
- But squaring also changes the units — squaring "dollars" gives you "dollars squared," which means nothing intuitively
- Standard deviation takes the square root of variance specifically to undo that squaring, bringing the units back to something meaningful

Why variance squares deviations (simplified idea)

`deviation = value - mean`

`squared_deviation = deviation 2` *# always positive*

Standard deviation undoes the squaring

`standard_deviation =  $\sqrt{\text{variance}}$`

This is previewed here — full detail comes in Descriptive Statistics



Scientific Notation — Taming Very Big & Very Small Numbers



**One digit,
times a
power of 10**

- Scientific notation writes any number as a single digit (1–9), times 10 raised to some power — a compact way to handle numbers that would otherwise need a wall of zeros
- A positive exponent means a very large number; a negative exponent means a very small one, closer to zero
- Datasets, probabilities, and computed error values constantly produce numbers like 0.0000031 or 4,500,000,000 — scientific notation keeps these genuinely readable
- Python itself displays numbers this way automatically once they get large or small enough — you'll see it constantly once real data shows up



Reading & Converting Scientific Notation

- The exponent tells you exactly which direction, and how far, to move the decimal point
- A positive exponent moves the decimal point right (making the number bigger); a negative exponent moves it left (making it smaller)
- Python writes this using "e" notation — 3.1×10^{-6} means exactly the same thing as 3.1×10^{-6}

$3.5 \times 10^6 = 3,500,000$ (decimal moves 6 places right)

$3.5 \times 10^{-6} = 0.0000035$ (decimal moves 6 places left)

Python's "e" notation — identical meaning

`3.5e6` # 3500000.0

`3.5e-6` # 0.0000035

You will see this constantly in real datasets and error values



Practice Set — Powers, Roots & Scientific Notation

Try these yourself first — that's the point

- 1 Calculate 3^5 and 2^{-3} by hand, without a calculator
- 2 Calculate $\sqrt{144}$ and $\sqrt[3]{64}$
- 3 Rewrite $\sqrt{50}$ as a fractional exponent expression
- 4 Convert 0.00042 into scientific notation, and 6.1×10^4 into ordinary decimal form
- 5 Explain in your own words why standard deviation takes a square root, but variance doesn't

Essential Math Conventions for Data Science

Small rules and symbols you'll need to read, calculate, and report data correctly



Rounding: Reporting the Right Amount of Precision

- Not every digit deserves to be reported
- Rounding means reducing a number to fewer digits while keeping it close to its original value.
 - The basic rule:
 - Look at the first digit you are removing
 - 0–4 → keep the rounding digit unchanged
 - 5–9 → increase the rounding digit by 1
 - Do the rounding at the end, not during intermediate calculations
- Rule of thumb:
 - Calculate with full precision → round when you report.

42.736

To 1 decimal place:

42.7 ← next digit is 3

To 2 decimal places:

42.74 ← next digit is 6

To 3 decimal places:

42.736



Significant Figures: How Precise Is Your Number?

- Significant figures tell the reader how much precision the number actually communicates
- The basic rules
 - Non-zero digits are always significant
 - Zeros between significant digits are significant
 - Leading zeros are not significant
 - Trailing zeros after a decimal point are significant
- Why should you care?
 - Reporting too many digits can create a false impression of precision.
 - Model accuracy = 87.436291%

Decimal places and significant figures are not the same thing.

42.7 → 3 significant figures

0.00427 → 3 significant figures

4270 → potentially ambiguous

4.27×10^3 → clearly 3 significant figures



Factorial Notation: $n!$

- A compact way to multiply all positive integers up to n
- You will see $!$ attached to numbers in probability and counting formulas.
- $n! = n \times (n - 1) \times (n - 2) \times \dots \times 2 \times 1$ (read is n factorial)

$$3! = 3 \times 2 \times 1$$

$$= 6$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

$$= 120$$

$$7! = 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$$

$$= 5040$$

$$0! = 1$$



PART 4

Logarithms, Intuitively

The one topic on this list you didn't ask for — and the one that will save you real confusion later



What Is a Logarithm, Really?



**The inverse
of an
exponent**

- A logarithm answers the exact reverse question of an exponent: "how many times do I need to multiply this base by itself to reach this number?"
- $\log_2(8)$ asks "how many times do I multiply 2 by itself to get 8?" — the answer is 3, because $2^3 = 8$
- Logarithms and exponents are inverse operations, exactly like roots and powers, or addition and subtraction
- If exponents describe explosive growth, logarithms describe the reverse — compressing a huge range of numbers back down into something manageable



Log Rules Worth Actually Knowing

- These are the handful of log rules that actually show up in real data science work — nothing more is genuinely necessary at this stage
- The product rule is especially useful — it's exactly why probabilities are often summed as log-probabilities instead of multiplied directly
- log base 10 and the natural log (base e, written $\ln$) are the two you'll see by far the most often

The defining relationship

$\log_b(x) = y$ means $b^y = x$

Product rule: log of a product = sum of logs

$\log(a \times b) = \log(a) + \log(b)$

The two logs you'll see constantly:

$\log_{10}(x)$ # base 10, common in scientific notation contexts

$\ln(x)$ # natural log, base $e \approx 2.718$, extremely common in ML



Why Logarithms Matter in AI & Data Science



**Compressing
huge ranges
into view**

- Log-transforming data is a standard EDA technique — it compresses a heavily skewed range (like income or population) into something you can actually visualize meaningfully
- Log Loss, one of the most common ways to score a classification model, is built directly on logarithms — it penalizes confident-but-wrong predictions far more harshly than unsure ones
- Entropy, the concept behind decision trees choosing what to split on, is also defined using logarithms
- None of this requires you to compute a log by hand — the goal here is recognizing the word "log" later and knowing roughly why it's there



Real-World Use: Log Transforms & a First Look at Log Loss

- A log transform is often as simple as applying `log()` to an entire column of data before plotting or modeling it
- Log Loss compares a model's predicted probability against what actually happened — and the penalty grows sharply as a confident prediction turns out wrong
- This is a preview, not the full picture — Log Loss gets properly covered once you reach model evaluation in the ML phase

Log-transforming a skewed column (conceptual)

```
df["income_log"] = np.log(df["income"])
```

Log Loss (simplified, single prediction)

heavy penalty for being confident AND wrong

```
loss = -(actual × log(predicted) + (1-actual) × log(1-predicted))
```

Predicting 0.99 when the true answer was 0 -> a HUGE loss

Predicting 0.5 (unsure) when wrong -> a much smaller loss



Practice Set — Logarithms

Try these yourself first — that's the point

- 1 Calculate $\log_2(16)$ and $\log_{10}(1000)$ by reasoning through the definition, not a calculator
- 2 Explain in your own words why $\log(a \times b)$ equals $\log(a) + \log(b)$
- 3 Look up one real dataset column (income, population, or file sizes) and explain why a log transform might help visualize it
- 4 In your own words, explain why a confidently wrong prediction should be penalized more harshly than an unsure wrong one



PART 5

Reading Math Notation

The alphabet every formula from the next session onward is written in



Why Notation Feels Scary (But Genuinely Isn't)



**Symbols,
not
magic**

- Math notation is just a set of agreed-upon shorthand symbols — the same way `*args` and f-strings were unfamiliar shorthand the first time you saw Python
- A formula that looks intimidating as a block of symbols often says something completely ordinary once translated into plain English, one symbol at a time
- You don't need to memorize notation in the abstract — you need enough familiarity to translate it, slowly, the first few times you see it
- Every symbol on the next few slides will come back constantly starting in the very next session — this is worth sitting with



Variables & Function Notation

- A variable (x , y , n) is just a placeholder for a number whose exact value isn't fixed — exactly like a parameter in a Python function
- $f(x)$ means "a function called f , applied to input x " — it's notation, not multiplication; f isn't being multiplied by x
- This is precisely the same idea as a Python function — $f(x)$ in math and `def f(x):` in code are describing the exact same concept

<i># Math notation</i>	<i># The equivalent Python</i>
$f(x) = x^2$	<code>def f(x):</code> <code> return x ** 2</code>
$f(3) = 9$	<code>f(3)</code> <i># 9</i>
<i># $f(x)$ is read as "f of x" — never "f times x"</i>	



Summation Notation — Σ

- Σ (the Greek capital letter sigma) simply means "add up everything that follows, for every value in the given range"
- The small numbers above and below Σ tell you exactly where to start and stop — $i=1$ to n means "for every i from 1 through n "
- This one symbol will appear constantly starting next session — it's the entire notation behind the mean, and behind variance

$$\sum_{i=1}^n x_i = x_1 + x_2 + x_3 + \dots + x_n$$

Example: sum of [4, 7, 2] using summation notation

$$\sum x_i \text{ for } i=1 \text{ to } 3 = 4 + 7 + 2 = 13$$

In Python, this is exactly what sum() does

`sum([4, 7, 2]) # 13`



Subscripts & Indices

- A subscript (the small number after a variable) identifies WHICH item in a list or sequence you're referring to
- x_i means "the i-th value of x" — exactly the same idea as `x[i]` when indexing a Python list
- `n` almost always represents the total count of items — seeing "for `i = 1` to `n`" is math's way of saying "loop through every item"

Math notation *# The equivalent Python*

$x_1, x_2, x_3, \dots, x_n$ `x = [10, 20, 30, ..., x_n]`

x_i `x[i]`

`n = 5` `len(x)` # 5



Inequality & Other Common Symbols

- These symbols show up constantly in probability, statistics, and set notation — worth recognizing on sight
- $\approx$ specifically matters in data science, since almost nothing computed from real data comes out as a perfectly clean number
- Once these feel familiar, an intimidating-looking formula becomes a sentence you can actually read left to right

$\leq$ less than or equal to
 $\geq$ greater than or equal to
 $\neq$ not equal to
 $\approx$ approximately equal to
 $\in$ "is an element of" (belongs to a set)
 ∞ infinity
 $|x|$ absolute value of x (distance from zero, always positive)



Putting It Together: Reading the Mean Formula

- Every symbol on this slide has already appeared earlier in this session — this is genuinely just translation practice, not new material
- Read it exactly like this: "the mean, x-bar, equals one over n, times the sum of x-sub-i, for i from 1 to n"
- In plain English: "add up every value, then divide by how many values there are" — exactly the average you already knew how to compute

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Translated into Python, line by line:

```
def mean(x):  
    n = len(x)      # "n"  
    total = sum(x)   # "Σ xi for i=1 to n"  
    return total / n # "1/n × ..."
```



Practice Set — Reading Notation

Try these yourself first — that's the point

- 1 Translate $f(x) = 2x + 1$ into a working Python function, then evaluate $f(5)$ both on paper and in code
- 2 Write out, in plain English, what $\sum x_i$ for $i=1$ to 4 means for the list $[3, 6, 1, 8]$
- 3 Given $x = [12, 18, 24]$, identify x_1 , x_2 , and n
- 4 Translate the inequality $x \geq 10$ into a plain-English sentence
- 5 Without looking back, explain the mean formula from the last slide out loud, symbol by symbol



INDUSTRY-STYLE TASKS

Build the Fluency Every Formula From Here Depends On

Not abstract drills — these are the exact calculations behind real metrics, transforms, and formulas you'll use constantly



Today's Tasks — Part A: Percentages, Ratios, Powers & Roots

Small tasks — do them in order



1

A model's accuracy went from 68% to 79%. Calculate the percentage change, and separately, the raw percentage-point difference — explain why they're different numbers.



2

A dataset has a class imbalance ratio of 1:24 (rare class to common class). If the common class has 4,800 rows, calculate how many rows are in the rare class.



3

Calculate the standard deviation of the set [4, 4, 4] by hand (variance, then square root) and explain why the result makes intuitive sense.



Today's Tasks — Part B: Scientific Notation & Logarithms

Small tasks — do them in order



1

Convert 0.0000078 and 250,000,000 into scientific notation, and explain out loud which direction the decimal moved and why.



2

Calculate $\log_2(64)$ and $\log_{10}(10000)$ using the definition of a logarithm, without a calculator.



3

Find a real, publicly available dataset with a skewed numeric column, and describe in your own words how a log transform would change its shape.



Today's Tasks — Part C: Reading Notation

Small tasks — do them in order



1

Translate the function $f(x) = x^2 - 3$ into Python code, and confirm $f(4)$ matches by hand and by running the code.



2

Write out $\sum x_i$ for $i=1$ to 5 in plain English for the list $[2, 4, 6, 8, 10]$, then compute it both by hand and with Python's `sum()`.



3

Rewrite the mean formula from this session in your own words, then use it to calculate the mean of $[15, 22, 18, 9]$ by hand.



Put It All Together

Decode a real, published data-science result using every skill from this session



Mini Project: Decode a Real Data Science Result



**Every skill
from today,
combined**

- Find (or use a provided) real model report containing a sentence like: "Accuracy improved from 71.2% to 85.9%, precision reached 0.91, and the model's Log Loss dropped to 4.3×10^{-2} "
- Calculate the exact percentage change in accuracy, and explain what a precision of 0.91 means as a ratio of true positives to predicted positives
- Convert the scientific notation Log Loss value into ordinary decimal form, and explain in plain English why a LOWER Log Loss is the goal, unlike accuracy
- Write the whole result out as a short paragraph in plain English, as if explaining it to someone who has never seen a percent sign, a ratio, or scientific notation before



Percentages, ratios, powers, roots, logs, and notation aren't separate hurdles — they're the one shared vocabulary every formula from here on is written in.

Next session: Descriptive Statistics — describing what your data actually looks like